Marwadi University	**Marwadi University** **Faculty of Technology** **Department of Information and Communication Technology**	
Subject: Artificial Intelligence (01CT0616)	**Aim: To obtain the classification of various classes using k-Nearest Neighbor approach**	
Experiment No: 03	**Date:**	**Enrollment No: 92410133012**

Aim: To obtain the classification of various classes using k-Nearest Neighbor approach

IDE: Google Colab

Theory:

This algorithm is used to solve the classification model problems. K-nearest neighbor or K-NN algorithm basically creates an imaginary boundary to classify the data. When new data points come in, the algorithm will try to predict that to the nearest of the boundary line. Therefore, larger k value means smother curves of separation resulting in less complex models. Whereas, smaller k value tends to over fit the data and resulting in complex models. It's very important to have the right k-value when analyzing the dataset to avoid over fitting and under fitting of the dataset.

The model representation for KNN is the entire training dataset. It is as simple as that. KNN has no model other than storing the entire dataset, so there is no learning required. Efficient implementations can store the data using complex data structures like k-d trees to make look-up and matching of new patterns during prediction efficient. Because the entire training dataset is stored, you may want to think carefully about the consistency of your training data. It might be a good idea to curate it, update it often as new data becomes available and remove erroneous and outlier data.

K-NN algorithm assumes the similarity between the new case/data and available cases and put the new case into the category that is most similar to the available categories. K-NN algorithm stores all the available data and classifies a new data point based on the similarity. This means when new data appears then it can be easily classified into a well suite category by using K- NN algorithm. K-NN algorithm can be used for Regression as well as for Classification but mostly it is used for the Classification problems.

K-NN is a non-parametric algorithm, which means it does not make any assumption on underlying data. It is also called a lazy learner algorithm because it does not learn from the training set immediately instead it stores the dataset and at the time of classification, it performs an action on the dataset. KNN algorithm at the training phase just stores the dataset and when it gets new data, then it classifies that data into a category that is much similar to the new data.

Example: Suppose, we have an image of a creature that looks similar to cat and dog, but we want to know either it is a cat or dog. So for this identification, we can use the KNN algorithm, as it works on a similarity measure. Our KNN model will find the similar features of the new data set to the cats and dogs images and based on the most similar features it will put it in either cat or dog category.

Methodology:

1. Load the basic libraries and packages
2. Load the dataset
3. Analyse the dataset
4. Pre-process the data

5. Visualize the Data
6. Separate the feature and prediction value columns
7. Select the number K of the neighbors
8. Calculate the Euclidean distance of K number of neighbors
9. Take the K nearest neighbors as per the calculated Euclidean distance.
10. Among these k neighbors, count the number of the data points in each category.
11. Assign the new data points to that category for which the number of the neighbor is maximum.
12. Our model is ready.

Pre Lab Exercise:

a. What is K in KNN-approach?

K means the number of nearest neighbors used to make a prediction.
Example: If K = 3, the algorithm checks the 3 closest data points and decides the output based on them.

b. Which are the types of distance metrices?

Common distance metrics used in KNN:

- **Euclidean Distance** (most common)
- **Manhattan Distance**
- **Minkowski Distance**
- **Hamming Distance** (for categorical data)

c. What can be the criteria of selection of the value of K?

- Choose K based on **accuracy**
- Use **cross-validation**
- Avoid:
- Very small K → overfitting
- Very large K → underfitting
- Usually odd values (like 3, 5, 7) are preferred

d. What are the advantages of KNN algorithms?

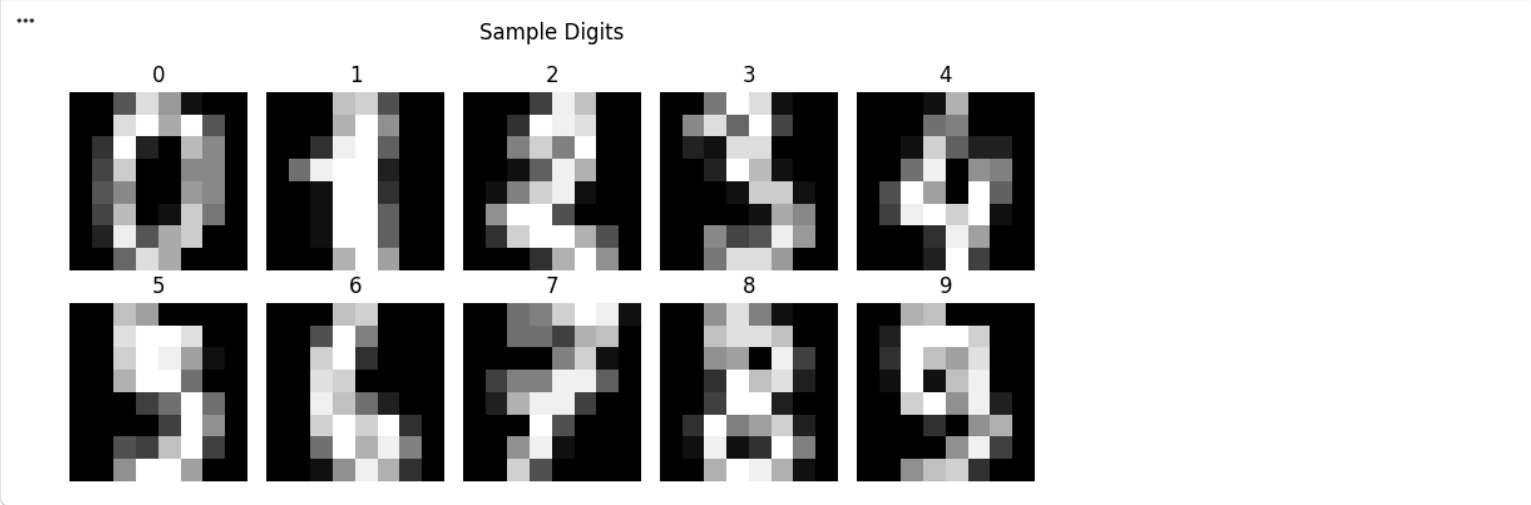
- Simple and easy to understand
- No training phase required
- Works well with small datasets
- Can be used for both classification and regression

Marwadi University	**Marwadi University** **Faculty of Technology** **Department of Information and Communication Technology**	
Subject: Artificial Intelligence (01CT0616)	**Aim: To obtain the classification of various classes using k-Nearest Neighbor approach**	
Experiment No: 03	**Date:**	**Enrollment No: 92410133012**

Program (Code):

```
# Importing required libraries
import time                    # To measure training and testing time
import numpy as np             # For numerical operations
import pandas as pd            # For data handling (tables)
import matplotlib.pyplot as plt # For plotting graphs
import seaborn as sns          # For advanced visualization

# Importing dataset and ML tools from sklearn
from sklearn.datasets import load_digits
from sklearn.model_selection import train_test_split, GridSearchCV
from sklearn.preprocessing import StandardScaler
from sklearn.pipeline import Pipeline

# Importing models
from sklearn.neighbors import KNeighborsClassifier
from sklearn.ensemble import RandomForestClassifier
from sklearn.neural_network import MLPClassifier

# Importing evaluation metrics
from sklearn.metrics import (
    accuracy_score,
    precision_score,
    recall_score,
    f1_score,
    confusion_matrix
)

# Loading dataset
digits = load_digits()      # Load digits dataset
X = digits.data             # Features (pixel values)
```

```
y = digits.target          # Labels (0–9)
images = digits.images     # Image format for visualization

# Displaying sample images
plt.figure(figsize=(8, 4))         # Set figure size
for i in range(10):                # Loop for 10 images
    plt.subplot(2, 5, i + 1)       # Create subplot (2 rows, 5 columns)
    plt.imshow(images[i], cmap="gray") # Show image in grayscale
    plt.title(f"{y[i]}")           # Show label as title
    plt.axis("off")                # Hide axis
plt.suptitle("Sample Digits")      # Overall title
plt.tight_layout()                 # Adjust layout
plt.show()                         # Display images
```




```
# Splitting dataset into training and testing
X_train, X_test, y_train, y_test = train_test_split(
    X, y, test_size=0.2, random_state=42
)

# KNN Pipeline (Scaling + Model)
knn_pipeline = Pipeline([
```

```
    ("scaler", StandardScaler()),   # Normalize data
    ("knn", KNeighborsClassifier()) # KNN model
])

# Parameters for KNN tuning
knn_param_grid = {
    "knn__n_neighbors": [3, 5, 7, 9, 13, 15, 20]
}
# Random Forest Pipeline
rf_pipeline = Pipeline([

    ("rf", RandomForestClassifier(random_state=42))
])

# Parameters for Random Forest tuning
rf_param_grid = {
    "rf__n_estimators": [25, 50, 100, 200, 500],
    "rf__max_depth": [None, 10, 20]
}
# MLP (Neural Network) Pipeline
mlp_pipeline = Pipeline([
    ("scaler", StandardScaler()),   # Normalize data
    ("mlp", MLPClassifier(
        max_iter=1000,            # Max iterations
        random_state=42,
        early_stopping=True        # Stop early if no improvement
    ))
])
# Parameters for MLP tuning
mlp_param_grid = {
    "mlp__hidden_layer_sizes": [(64,), (128,), (64, 64)],
    "mlp__activation": ["relu", "tanh"],
```

```
    #"mlp__alpha": [0.0001, 0.001],  # Regularization (commented)
    "mlp__learning_rate_init": [0.001, 0.01, 1.0, 5.0, 10.0]
}
# Function to train model and measure time
def train_with_timing(pipeline, param_grid):
    start = time.time()   # Start timer
    grid = GridSearchCV(
        pipeline,        # Model pipeline
        param_grid,      # Parameters to test
        cv=5,            # 5-fold cross validation
        scoring="f1_weighted", # Evaluation metric
        n_jobs=-1        # Use all CPU cores
    )
    grid.fit(X_train, y_train)  # Train model

    train_time = time.time() - start  # Calculate training time
    return grid, train_time          # Return model + time

# Training models
knn_grid, knn_train_time = train_with_timing(knn_pipeline, knn_param_grid)
rf_grid, rf_train_time = train_with_timing(rf_pipeline, rf_param_grid)
mlp_grid, mlp_train_time = train_with_timing(mlp_pipeline, mlp_param_grid)

# Function to measure prediction (inference) time
def inference_time(model):
    start = time.time()        # Start timer
    model.predict(X_test)      # Predict test data
    return time.time() - start # Return time taken

# Calculating inference time
knn_test_time = inference_time(knn_grid.best_estimator_)
rf_test_time = inference_time(rf_grid.best_estimator_)
```

```
mlp_test_time = inference_time(mlp_grid.best_estimator_)
# Function to evaluate model performance
def evaluate_model(model):

    y_pred = model.predict(X_test)  # Predict output

    return {
        "Accuracy": accuracy_score(y_test, y_pred),
        "Precision": precision_score(y_test, y_pred, average="weighted"),
        "Recall": recall_score(y_test, y_pred, average="weighted"),
        "F1-score": f1_score(y_test, y_pred, average="weighted")
    }

# Getting metrics for each model
knn_metrics = evaluate_model(knn_grid.best_estimator_)
rf_metrics = evaluate_model(rf_grid.best_estimator_)
mlp_metrics = evaluate_model(mlp_grid.best_estimator_)

# Creating result table
results_df = pd.DataFrame(
    [knn_metrics, rf_metrics, mlp_metrics],
    index=["KNN", "Random Forest", "MLP"]
)

# Adding training time
results_df["Training Time (s)"] = [
    knn_train_time, rf_train_time, mlp_train_time
]

# Adding inference time
results_df["Inference Time (s)"] = [
    knn_test_time, rf_test_time, mlp_test_time
```

]

print(results_df) # Display results

```
...                 Accuracy  Precision    Recall  F1-score  Training Time (s)  \
    KNN             0.969444   0.969666  0.969444  0.969287           5.000380
    Random Forest   0.977778   0.978417  0.977778  0.977820          49.642176
    MLP             0.980556   0.980832  0.980556  0.980577          58.312733

                    Inference Time (s)
    KNN                       0.019512
    Random Forest             0.106671
    MLP                       0.003206
```

#

Plotting performance comparison

results_df.iloc[:, :4].plot(kind="bar", figsize=(10, 6))

plt.title("Model Performance Comparison (Digits)")

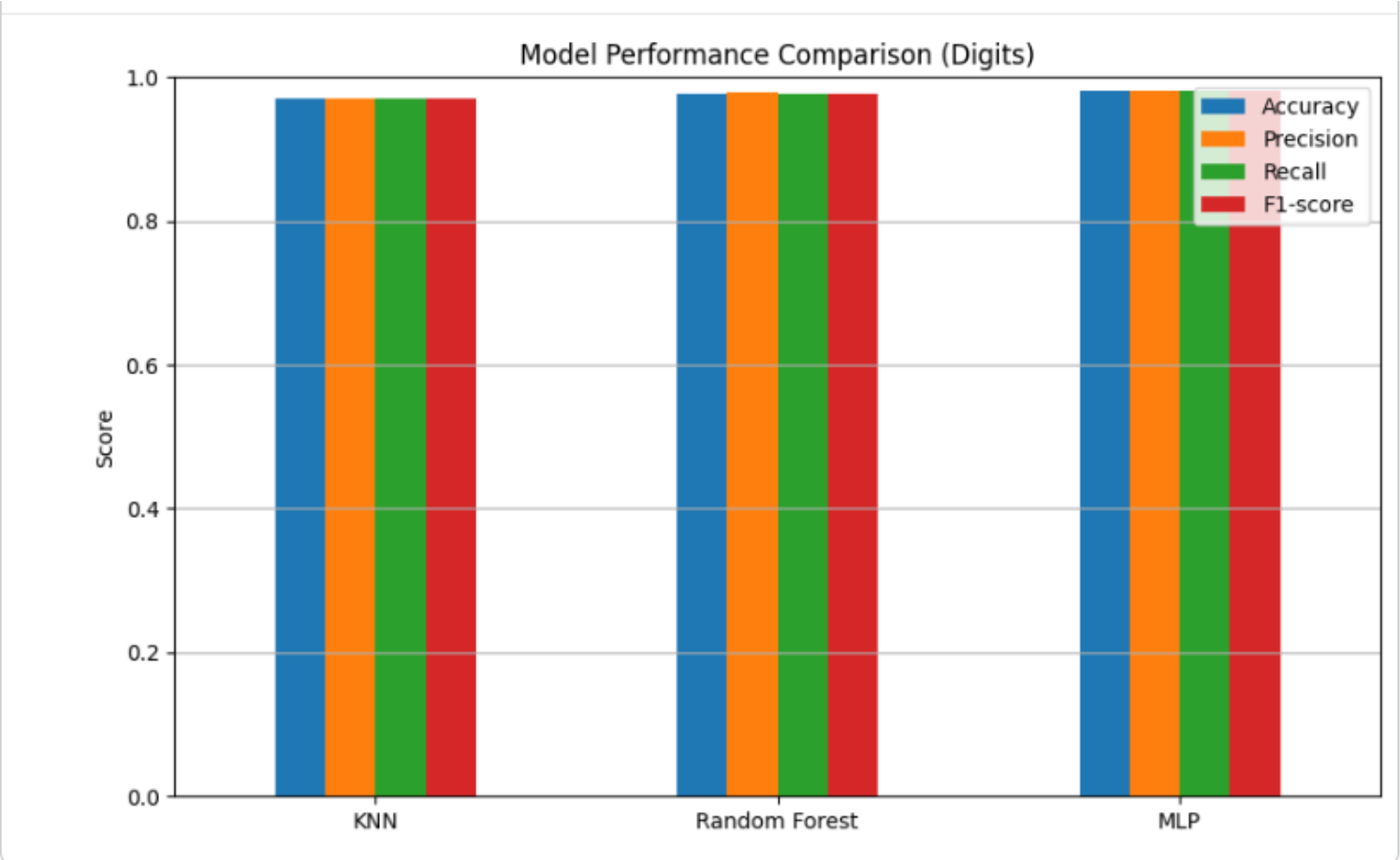
plt.ylabel("Score")

plt.ylim(0, 1)

plt.xticks(rotation=0)

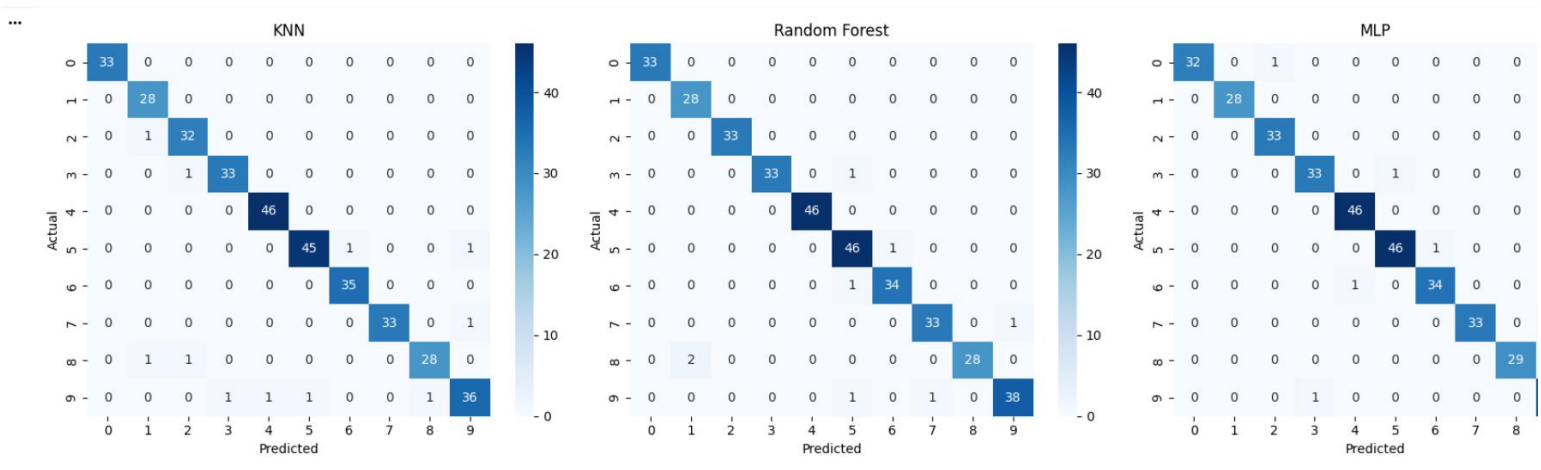
plt.grid(axis="y")

plt.show()




```
# Storing models in dictionary
models = {
    "KNN": knn_grid.best_estimator_,
    "Random Forest": rf_grid.best_estimator_,
    "MLP": mlp_grid.best_estimator_
}

# Plotting confusion matrices
fig, axes = plt.subplots(1, 3, figsize=(18, 5))

for ax, (name, model) in zip(axes, models.items()):
```

```
    cm = confusion_matrix(y_test, model.predict(X_test)) # Generate matrix
    sns.heatmap(cm, annot=True, fmt="d", cmap="Blues", ax=ax) # Plot heatmap
    ax.set_title(name)

    ax.set_xlabel("Predicted")
    ax.set_ylabel("Actual")
plt.tight_layout()
plt.show()
```



Marwadi University	**Marwadi University** **Faculty of Technology** **Department of Information and Communication Technology**	
Subject: Artificial Intelligence (01CT0616)	**Aim: To obtain the classification of various classes using k-Nearest Neighbor approach**	
Experiment No: 03	**Date:**	**Enrollment No: 92410133012**

Results:

To be attached with

a. **Confusion matrix**

- Shows **correct and incorrect predictions**
- Helps understand model performance
- Correct predictions (diagonal)
- Wrong predictions (off-diagonal)

b. **Accuracy Score**

- Measures **how many predictions are correct**
- Value between 0 to 1

Observation and Result Analysis:

a. **Nature of the dataset**

The dataset used is a **digits dataset** containing handwritten numbers from 0 to 9. It is a **multi-class classification dataset** where each image is converted into numerical pixel values. The dataset is small in size, clean, and suitable for testing machine learning algorithms.

b. **During Training Process**

During training, different models (KNN, Random Forest, and MLP) are trained using **GridSearchCV**, which tries multiple parameter combinations to find the best one.

- KNN does not actually learn but stores data
- Random Forest builds multiple decision trees
- MLP trains a neural network using iterations
  Training time varies: MLP takes more time, KNN takes less.

c. **After the training Process**

After training, the best model is selected based on performance metrics. The models are then tested on unseen data.

- Random Forest and MLP generally give **higher accuracy**
- KNN gives good results but depends on the value of K
- Performance is evaluated using accuracy, precision, recall, and F1-score

d. **Observation over the Learning Curve**

The learning behavior shows that:

- Small K in KNN may cause **overfitting**
- Large K may cause **underfitting**
- MLP improves gradually with training but takes time
- Random Forest gives **stable and balanced performance**
  Overall, proper parameter tuning is important for best results.

Marwadi University	**Marwadi University** **Faculty of Technology** **Department of Information and Communication Technology**	
Subject: Artificial Intelligence (01CT0616)	**Aim: To obtain the classification of various classes using k-Nearest Neighbor approach**	
Experiment No: 03	**Date:**	**Enrollment No: 92410133012**

Post Lab Exercise:

a. **Is KNN useful in regression-based problems?**

Yes, KNN can also be used for regression problems. Instead of majority voting, it predicts the output by taking the **average of nearest neighbors**.

b. **Why KNN is a non-parametric algorithm?**

KNN is called non-parametric because it does not assume any fixed mathematical formula or distribution. It directly uses the data to make predictions.

c. **What are the assumptions of KNN approach?**

- Similar data points are located close to each other
- Distance between points is meaningful
- Data is properly scaled for better accuracy

d. **How can the KNN approach make the prediction of the unseen dataset?**

KNN finds the **K nearest data points** from the training dataset using a distance metric.

- For classification → uses majority voting
- For regression → uses average value

e. **Why is KNN called a Lazy Learner?**

KNN is called a lazy learner because it does not build a model during training. It simply stores the data and performs computation only at the time of prediction.

f. **Why should we not use KNN for large dataset?**

KNN is not suitable for large datasets because:

- It requires **high computation time** for prediction
- It stores all data → high memory usage
- Slow performance due to distance calculations